

TP3 — Livraison progressive et observabilité

DevHub Campus SRE — Argo Rollouts, Prometheus, SLOs

Cheikhaoui Saad

Paul Blanchet

11 juillet 2026

Table des matières

Contexte	1
Partie I — Observabilité	2
SLI / SLO / error budget	2
Stack d’observabilité (kube-prometheus-stack via ArgoCD)	2
ServiceMonitor et dashboards Grafana	2
Partie II — Livraison progressive	3
Du Deployment au Rollout (canary)	3
Pilotage manuel (pause / promote / abort)	3
AnalysisTemplate — la promotion sur preuve	3
Blue/Green (planning)	4
Routage avancé par en-tête HTTP	4
Partie III — Alerting, comparatif et synthèse	4
PrometheusRule et Alertmanager	4
Argo Rollouts vs Flagger vs RollingUpdate natif	5
Ce que cette chaîne ne sait pas encore faire	5
Position d’architecte	5
Conclusion	5

Contexte

Troisième et dernière partie du cours Kubernetes ESGI 5 SRC. Objectif : instrumenter les trois services de la plateforme *DevHub Campus* (annuaire, planning, notif), déployer une stack d’observabilité via GitOps, puis brancher Argo Rollouts sur Prometheus pour des canaries et bascules blue/green auto-promus ou auto-rollbackés sur preuve métrique.

- **Dépôt** : github.com/chesaad/pulse-campus
 - **Cluster** : kind (réutilisé du TP2, ArgoCD déjà opérationnel)
 - **Outils installés** : `kubectl-argo-rollouts v1.9.0`, `promtool/amtool 3.13.1/0.28.0`
-

Partie I — Observabilité

SLI / SLO / error budget

Service	SLI clé	SLO	Error budget mensuel
annuaire	Disponibilité / Latence p95	99.5 % / < 300 ms	3h36
planning	Disponibilité / Latence p95 / succès création créneau	99.5 % / < 400 ms / 99 %	3h36 / 7h12
notif	Disponibilité / Latence p95 / succès création événement	99 % / < 200 ms / 99.5 %	7h12 / 3h36

Seuils choisis à 99.5 % (pas 99.9 %) : sur un cluster de TP, un SLO trop strict rend l'error budget en permanence négatif et donc inutile comme signal de décision. notif reste sur un SLO de disponibilité plus lâche car c'est le seul service resté en simple Deployment, sans filet canary.

Les buckets d'histogramme Prometheus de chaque service (`metrics.buckets`) sont calés sur ces seuils p95 (bucket exact à 300/400/200 ms), validés localement par `docker run + curl /metrics + promtool check metrics`.

Stack d'observabilité (kube-prometheus-stack via ArgoCD)

Prometheus, Alertmanager et Grafana sont déployés en GitOps (Application ArgoCD multi-source, chart + values versionnés). Points notables :

- Mot de passe Grafana **jamais en clair dans Git** — Secret Kubernetes créé hors dépôt, référencé via `admin.existingSecret`.
- `AppProject` restreint explicitement aux repos nécessaires et aux ressources cluster-scoped réellement utilisées (pas de wildcard `"*"`).
- Deux corrections nécessaires en pratique : le sous-composant `coreDns` du chart pose un Service dans `kube-system` (hors périmètre de l'`AppProject`, désactivé) ; le squelette de départ désactivait les *admission webhooks* de l'opérateur Prometheus, ce qui bloquait le pod en attente d'un Secret jamais créé (version de chart plus récente que prévue — réactivés).

Validation : 8 Applications ArgoCD Synced/Healthy, tous les targets Prometheus up.

ServiceMonitor et dashboards Grafana

Un ServiceMonitor par service (label `release: kps` requis pour être découvert par Prometheus — piège le plus fréquent signalé par le cours). Un dashboard Grafana par service (4 panneaux : RPS, taux d'erreur, latence p50/p95/p99, version active), templaté par variables `$service/$namespace`, exporté en JSON et déployé via ConfigMap.

Bug trouvé et corrigé : le nom du Service Kubernetes dépendait du nom de release ArgoCD (`annuaire-dev-annuaire` au lieu de `annuaire`), ce qui aurait rendu tout le PromQL du rapport

silencieusement vide. Corrigé en ajoutant le support standard `fullnameOverride` aux charts Helm.

Partie II — Livraison progressive

Du Deployment au Rollout (canary)

annuaire migré vers un Rollout Argo avec stratégie canary et routage réel au niveau requêtes via ingress-nginx (pas une simple approximation par ratio de replicas). Un canary réel a été observé de bout en bout : 20 % → pause → 50 % → pause → 100 %, avec le poids canary-weight de l'Ingress auto-générée confirmé identique au SetWeight du Rollout à chaque palier.

Trois difficultés réelles rencontrées et corrigées :

1. L'ancien Deployment non supprimé du cluster (politique prune: false) entrainait en conflit avec le nouveau Rollout.
2. Une Application ArgoCD restait OutOfSync en permanence à cause d'un champ par défaut (protocol: TCP) non normalisé sur une CRD.
3. Même symptôme après un premier canary réussi : Argo Rollouts patche dynamiquement le sélecteur des Service stable/canary, un champ que Git ne peut pas connaître à l'avance — résolu avec ignoreDifferences.

Pilotage manuel (pause / promote / abort)

Trois scénarios démontrés en conditions réelles :

- **Promotion normale** : promote fait sauter la pause manuelle au palier suivant.
- **Annulation (abort)** : le poids canary retombe à 0 dans le cluster, mais **Git n'est pas modifié automatiquement** — git revert nécessaire pour réaligner l'état déclaré sur l'état réel.
- **Promotion forcée (promote --full)** : saute tous les paliers restants — acceptable seulement en vraie urgence documentée, jamais en routine.

AnalysisTemplate — la promotion sur preuve

Objectif central du TP : brancher Argo Rollouts sur Prometheus pour qu'il décide seul de promouvoir ou d'annuler. Deux métriques (taux d'erreur < 1 %, latence p95 sous le SLO), échantillonnées 30 s × 10 mesures (5 minutes), une seule mesure en échec déclenche un rollback immédiat.

Trois bugs PromQL corrigés avant d'obtenir un AnalysisTemplate fonctionnel — le plus instructif : tant qu'aucune erreur 5xx n'a jamais été vue, la série Prometheus correspondante n'existe **pas du tout** (résultat vide, pas zéro), ce qui faisait planter le fournisseur Prometheus d'Argo Rollouts. Corrigé avec l'idiome PromQL `OR on() vector(0)`.

Résultat démontré en direct : un canary sain a été promu de 25 % à 100 % **entièrement automatiquement**, sans aucune commande manuelle — exactement la promesse du cours.

Blue/Green (planning)

planning migré en stratégie blue/green (activeService/previewService, ancienne version conservée 5 minutes après bascule pour rollback instantané). Les deux versions tournent simultanément à pleine capacité (coût ressources doublé pendant la transition).

Anomalie rencontrée, documentée honnêtement plutôt que masquée : la bascule automatique s’est produite à chaque nouvelle révision malgré `autoPromotionEnabled: false`, quel que soit le scénario testé (4 configurations essayées). Cause précise non élucidée dans le temps disponible — hypothèse la plus probable : interaction entre `ignoreDifferences` (nécessaire par ailleurs) et la lecture par le contrôleur de son propre état précédent.

Routage avancé par en-tête HTTP

Une requête portant `X-Beta-User: true` atteint systématiquement le canary, indépendamment du poids courant — confirmé par les logs d’accès ingress-nginx (upstream canary explicitement identifié). Permettrait à l’équipe produit de tester une release sur ses propres comptes avant tout utilisateur, en complément (pas en remplacement) de la validation automatique par métriques.

Partie III — Alerting, comparatif et synthèse

PrometheusRule et Alertmanager

Deux familles d’alertes, gradées par sévérité : `HighErrorRate` (> 1 % sur 5 min, sévérité *page*, réveille) et `LatencyDegraded` (p95 dégradé sur 30 min, sévérité *ticket*, attend). Alertmanager route chaque sévérité vers un webhook distinct avec un `repeat_interval` différencié (1h/12h) pour éviter le spam. Configuration validée hors-cluster (`promtool check rules`, `amtool check-config`) puis confirmée chargée en direct dans le pod Alertmanager.

Les notifications Argo Rollouts (succès/échec de rollout vers un webhook dédié) ont été correctement configurées côté GitOps mais leur livraison échoue en pratique (bug de résolution d’URL vide, reproductible sur plusieurs configurations testées) — limitation documentée plutôt que masquée.

Argo Rollouts vs Flagger vs RollingUpdate natif

Critère	RollingUpdate	Argo Rollouts	Flagger
Courbe d'apprentissage	5	2	2
Intégration ArgoCD	5	3	3
Stratégies (canary/blueGreen/A-B)	0	4	4
Metric providers	0	5	4
Dashboard natif	0	4	1
Coût opérationnel	5	3	4
Adapté à un service mesh	1	3	5
Risque si le contrôleur tombe	5	2	2

Argo Rollouts s'impose pour la cohérence d'outillage avec ArgoCD et la richesse des metric providers ; Flagger reste préférable dans un contexte Flux ou service mesh déjà en place.

Ce que cette chaîne ne sait pas encore faire

Sept limites structurelles identifiées, chacune avec un risque concret et un outil complémentaire : traçabilité distribuée (OpenTelemetry/Jaeger), logs centralisés corrélés (Loki), mesure côté utilisateur réel (RUM), chaos engineering (Chaos Mesh), politique d'admission des manifestes (Kyverno), signature et provenance des images (Sigstore/SLSA), sauvegarde et disaster recovery applicatif (Velero).

Position d'architecte

En tant que responsable plateforme sur une organisation à 10 services et 30 développeurs, la chaîne GitOps + Argo Rollouts + Prometheus constituerait le socle minimal conservé intégralement — rien dans ce TP n'a semblé relever du sur-engineering à cette échelle. Priorités d'ajout, dans l'ordre : traçabilité distribuée dès qu'un service en appelle un autre, logs centralisés corrélés, puis une politique d'admission (Kyverno) empêchant de contourner Rollout/AnalysisTemplate par erreur humaine — seule protection de la liste contre une régression *humaine* plutôt que technique. Le chaos engineering et la signature d'images viennent ensuite. Limite assumée : cette chaîne ne protège toujours rien côté base de données, prochain chantier avant un premier client réel.

Conclusion

L'ensemble de la chaîne a été implémenté et validé sur un cluster réel (pas seulement écrit) : 8 Applications ArgoCD Synced/Healthy, canary et blue/green démontrés en conditions réelles, promotion et rollback automatiques observés en direct. Deux anomalies (bascule blue/green automatique, livraison des notifications Rollouts) ont été rencontrées, diagnostiquées en profondeur et documentées honnêtement plutôt que dissimulées — dans l'esprit du TP : mesurer et rapporter avec rigueur, y compris quand le résultat n'est pas celui attendu.